Step 19 - Shared Counter and SMP Systems 1
Reference Note 1
Learning Objectives 1
1. What is SMP? 2
2. Timing Problem in SMP Systems 2
Example Problem 3
3. ARM Solution 3
Architecture View 3
4. Shared Timeline 4
5. Cross-Core Timestamping 4
Core 0 4
Core 3 5
Benefits 5
6. Task Migration 5
Example 6
Before Migration 6
After Migration 6
7. Shared Counter vs Per-Core Timers 6
Shared Resource 6
Per-Core Resources 7
Architecture Summary 7
8. Example System 7
Four-Core System 8
Conceptual View 8
9. Why Not Use Per-Core Counters? 8
10. Relationship to Virtualization 8
Example 9
Architecture Summary 9
Common Misconceptions 10

Misconception 1 10
Misconception 2 10
Misconception 3 10
Misconception 4 10
Key Takeaways 11
Knowledge Check 11
Q1 11
Q2 11
Q3 11
Q4 11
Q5 12
Answer Key 12

Step 19 - Shared Counter and SMP Systems

Reference Note

This chapter is based on concepts described in the ARM Generic Timer Architecture documentation.

Previous chapters explained:

- System Counter
- Physical Counter
- Virtual Counter
- Physical and Virtual Timers
- Interrupt delivery

This chapter explains why ARM chose:

One Shared System Counter

instead of:

One Counter Per CPU Core

and how this design simplifies timing in multicore (SMP) systems.

Learning Objectives

After completing this chapter, you should understand:

- What SMP means
- Why ARM uses a shared System Counter
- How all cores observe the same timeline
- Benefits for task migration
- Benefits for timestamping
- Relationship between shared counters and per-core timers

1. What is SMP?

SMP stands for:

Symmetric Multi-Processing

Examples:

System	CPU Cores
Cortex-A53 Cluster	4
Cortex-A72 Cluster	4
Cortex-A78 Cluster	8
Neoverse Systems	Many

In an SMP system:

In OS terminology, **SMP = Symmetric Multiprocessing**.

It is a system architecture where:

- Multiple CPU cores (or processors) are treated equally.

- All CPUs share the same main memory.
- Any CPU can execute operating system code, handle interrupts, and run application threads.
- The scheduler can move thread between CPUs to balance the load.

In an SMP system:

- A thread may start on Core 0.
- Later the scheduler may move it to Core 2.
- All cores see the same physical memory.
- All cores can read the same system counter (`CNTPCT_EL0` / `CNTVCT_EL0`).

2. Timing Problem in SMP Systems

Suppose ARM had designed:

Core	Counter
Core 0	Counter A
Core 1	Counter B
Core 2	Counter C
Core 3	Counter D

Potential problems:

- Counters drift apart
- Timestamps cannot be compared reliably
- Task migration becomes difficult
- Synchronization becomes complicated

Example Problem

Suppose:

Core	Counter Value
Core 0	1,000,000

Core 1	999,500

Now:

Which timestamp is newer?

The answer becomes unclear.

3. ARM Solution

ARM uses:

One Shared System Counter

for all cores.

Architecture View

Shared Resource	Used By
System Counter	Core 0
System Counter	Core 1
System Counter	Core 2
System Counter	Core 3

Every core observes:

The same timeline

4. Shared Timeline

Suppose the System Counter value is:

1,000,000

Then:

Core	Counter Read
Core 0	~1,000,000
Core 1	~1,000,000
Core 2	~1,000,000
Core 3	~1,000,000

Minor differences may appear due to:

Instruction execution latency

but all cores observe the same underlying counter.

5. Cross-Core Timestamping

Suppose:

Core 0

Records event:

Timestamp = 1,000,000

Core 3

Later records event:

Timestamp = 1,100,000

Conclusion:

Event on Core 3 happened later.

This comparison is possible because both timestamps originate from the same System Counter.

Benefits

Feature	Benefit
Common timestamps	Easy event ordering
Shared timeline	Reliable measurements
Inter-core synchronization	Simpler software

6. Task Migration

A task can move between cores.

Example:

Time	Core
t0	Core 0
t1	Core 2

Without a shared counter:

Task would see different clocks.

With the ARM design:

Time appears continuous.

Example

Before Migration

Core	Counter
Core 0	1,000,000

After Migration

Core	Counter
Core 2	1,000,500

The task experiences:

Normal time progression.

No special synchronization is required.

7. Shared Counter vs Per-Core Timers

Many engineers confuse these concepts.

Shared Resource

Component	Scope
System Counter	Entire System

There is only:

One System Counter

Per-Core Resources

Component	Scope
Physical Timer	Per Core
Virtual Timer	Per Core
Hypervisor Timer	Per Core
PPIs	Per Core

Architecture Summary

Resource Type	Example
Shared	System Counter
Shared	CNTFRQ_EL0
Per-Core	Physical Timer
Per-Core	Virtual Timer
Per-Core	Hypervisor Timer
Per-Core	Timer PPIs

8. Example System

Four-Core System

Resource	Quantity
System Counter	1
Physical Counters	Views of Same Counter
Physical Timers	4
Virtual Timers	4
PPIs	Per Core

Conceptual View

Component	Number
Shared Timeline	1
Timer Instances	One Per Core

9. Why Not Use Per-Core Counters?

Potential issues:

Problem	Description
Drift	Counters diverge
Synchronization	Difficult
Event Ordering	Ambiguous
Migration	Complex
Time Measurement	Inconsistent

ARM avoids all of these by using:

One shared System Counter

10. Relationship to Virtualization

Even virtualized systems rely on:

One System Counter

The Hypervisor creates:

Different views of time

using:

CNTVOFF_EL2

but the underlying source remains shared.

Example

Component	Value
System Counter	Shared
Guest A Offset	Applied
Guest B Offset	Applied

Result:

Different virtual timelines

Same physical time source

Architecture Summary

Component	Purpose
System Counter	Common notion of time
Shared Timeline	Visible to all cores
Physical Timer	Per-core event generation
Virtual Timer	Per-core virtual events
Task Migration	Time continuity
Timestamping	Consistent event ordering

Common Misconceptions

Misconception 1

Each CPU has its own System Counter

False.

The System Counter is shared.

Misconception 2

Shared Counter means shared timer

False.

Timers remain per-core.

Misconception 3

Task migration requires time synchronization

False.

The shared counter already provides a common timeline.

Misconception 4

Virtualization creates a new hardware counter

False.

Virtual time is derived from the same System Counter.

Key Takeaways

Concept	Key Point
SMP	Multiple CPUs share one timeline
System Counter	Shared by all cores
Timers	Per-core resources
Timestamping	Consistent across cores
Task Migration	Seamless time continuity
Virtualization	Multiple views of one time source

Knowledge Check

Q1

Why does ARM use one shared System Counter?

Q2

What problem would independent per-core counters create?

Q3

Is the Physical Timer shared between cores?

Q4

Why is task migration simple with a shared counter?

Q5

Does virtualization require additional hardware counters?

Answer Key

Question	Answer
Q1	To provide a common notion of time across all cores
Q2	Drift, synchronization problems, and inconsistent timestamps
Q3	No, it is per-core
Q4	All cores observe the same timeline
Q5	No, it uses one shared System Counter